Database Complete Interview Reference — Full Edition

> **All topics merged:** Core Concepts · SQL · Indexing · Transactions · Concurrency · Distributed Systems · NoSQL · Performance · Security · Cloud · Advanced Every section includes subtopics, keywords, and interview notes.

TABLE OF CONTENTS

Foundations

- 1. Core Database Concepts
- 2. Data Models
- 3. Database Types
- 4. Data Types
- 5. Database Objects

Relational & SQL

- 6. Relationships & Constraints
- 7. SQL & Querying
- 8. Normalization & Schema Design
- 9. Indexing
- 10. Query Optimization

Transactions & Concurrency

- 11. Transactions
- 12. Concurrency Control
- 13. Consistency Models

Storage & Internals

- 14. Storage Engine & Architecture
- 15. Database Internals (Deep Dive)

Distribution & Scaling

- 16. Data Distribution & Partitioning
- 17. Replication
- 18. Distributed Databases
- 19. Performance & Scaling

Specialized Databases

- 20. NoSQL Systems
- 21. Graph Databases
- 22. Time-Series Databases
- 23. Vector Databases & AI/ML Data

Data Engineering

- 24. Data Warehousing & Analytics
- 25. Streaming & Real-Time Data
- 26. Schema Migration & Evolution
- 27. Multi-Model & Polyglot Persistence

Operations & Reliability

- 28. Backup & Recovery
- 29. Observability & Database Monitoring
- 30. Security
- 31. Compliance & Data Regulations

Cloud & Advanced

- 32. Cloud Databases
- 33. Database Tools & Ecosystem
- 34. Anti-Patterns
- 35. Advanced / Staff+ Level Topics

- 36. Interview Master Keywords

---

1. Core Database Concepts

Fundamentals

- **Database** → Organized collection of structured data stored and accessed electronically.
- **DBMS** → Software managing storage, retrieval, and manipulation (MySQL, PostgreSQL, Oracle).
- **RDBMS** → DBMS based on the relational model; tables with rows/columns; SQL interface.
- **NoSQL** → Non-relational; flexible schema; horizontally scalable; key-value/document/graph/column stores.
- **Data Model** → Abstract representation of how data is organized and related.
- **Schema** → Blueprint defining structure (tables, columns, types, constraints). Logical blueprint of the DB.
- **Instance** → Snapshot of actual data in the DB at a given moment (schema = type, instance = value).
- **Metadata** → Data about data; describes structure, format, relationships.
- **Data Dictionary** → Centralized metadata repository; tracks table names, column types, constraints.
- **Catalog** → System-maintained metadata store (e.g. `pg_catalog` in PostgreSQL).
- **Logical vs Physical Model** → Logical = entities/relationships (ER diagram); Physical = actual tables/indexes/storage.
- **Keywords**: Schema vs Instance is like a class vs object; DBMS enforces schema; NoSQL may be schema-less or schema-flexible.

---

2. Data Models

Types of Data Models

- **Relational Model** → Tables with rows/columns; relationships via foreign keys; basis of all RDBMS.
- **Hierarchical Model** → Tree structure; parent-child; fast reads but rigid (IBM IMS legacy).
- **Network Model** → Graph of records; many-to-many via set; predecessor to relational.
- **Document Model** → Semi-structured JSON/BSON documents; flexible schema (MongoDB, CouchDB).
- **Key-Value Model** → Simplest; key maps to opaque value; ultra-fast (Redis, DynamoDB).

- **Column-Family Model** → Rows grouped into column families; sparse; write-optimized (Cassandra, HBase).
- **Graph Model** → Nodes + edges; traversal queries; relationships are first-class (Neo4j, Neptune).
- **Time-Series Model** → Optimized for timestamped data; append-heavy; time as primary index (InfluxDB, TimescaleDB).
- **Object-Oriented Database** → Objects stored directly; rare; used in CAD/scientific apps.
- **Vector Model** → Stores high-dimensional embeddings; optimized for similarity search (Pinecone, Weaviate).
- **Keywords**: Document = flexible hierarchy; Column-family = analytical wide rows; Graph = relationship-heavy queries; Vector = AI/ML similarity.

---

3. Database Types

Classification by Workload & Architecture

- **OLTP** → Online Transaction Processing; high-frequency short transactions; row-store; normalized (MySQL, PostgreSQL).
- **OLAP** → Online Analytical Processing; complex aggregations over large datasets; column-store (Redshift, BigQuery).
- **HTAP** → Hybrid Transactional/Analytical Processing; handles both workloads (TiDB, SingleStore).
- **In-Memory Database** → Data stored in RAM; microsecond latency; volatile or persisted (Redis, VoltDB, SAP HANA).
- **Distributed Database** → Data spread across nodes; horizontal scale; CAP trade-offs (CockroachDB, Spanner).
- **Embedded Database** → Runs inside application process; no separate server (SQLite, H2, RocksDB).
- **Cloud Database** → Managed by cloud provider; auto-scale, automated backups (RDS, Aurora, Cloud SQL).
- **Serverless Database** → Scale-to-zero; pay-per-query; no server management (Aurora Serverless, Neon, PlanetScale).
- **NewSQL** → SQL interface + horizontal scalability + ACID; addresses RDBMS scaling limits (CockroachDB, YugabyteDB).
- **Keywords**: OLTP = many small writes; OLAP = few large reads; choose DB type based on read/write ratio and query patterns.

---

4. Data Types

Common Data Types Across Databases

- **Numeric** → `INT`, `BIGINT`, `DECIMAL(p,s)`, `FLOAT`, `NUMERIC`; choose precision carefully for money (use DECIMAL not FLOAT).
- **Character / String** → `CHAR(n)` fixed-width, `VARCHAR(n)` variable; `TEXT` unlimited; CHAR pads, VARCHAR doesn't.
- **Boolean** → `BOOLEAN`; true/false/null; stored as 1 byte in PostgreSQL.
- **Date / Time / Timestamp** → `DATE`, `TIME`, `TIMESTAMP`, `TIMESTAMPTZ` (with timezone); always store in UTC.
- **Binary (BLOB)** → Binary Large Object; store files/images; better to store path + object storage instead.
- **Text (CLOB)** → Character Large Object; large text; full-text search applicable.
- **JSON / JSONB** → Semi-structured; JSONB (PostgreSQL) is binary, indexed, queryable; JSON is text-stored.
- **Spatial / Geospatial** → `GEOMETRY`, `GEOGRAPHY`; PostGIS extension; latitude/longitude; distance queries.
- **Array / Composite** → PostgreSQL supports array columns and composite types natively.
- **UUID / GUID** → 128-bit unique ID; globally unique; good for distributed primary keys (random UUIDs cause index fragmentation).
- **Vector / Embedding** → Fixed-length float array; used for ML embeddings; pgvector extension in PostgreSQL.
- **Keywords**: DECIMAL over FLOAT for money; TIMESTAMPTZ over TIMESTAMP; JSONB over JSON for querying; UUID v7 preserves time-ordering.

---

5. Database Objects

Core Objects

- **Table** → Primary storage unit; rows + columns; lives inside a schema.
- **Row / Record** → Single tuple; represents one entity instance.
- **Column / Field** → Named attribute with a specific data type and constraints.
- **View** → Virtual table from a stored SELECT; no own storage; simplifies queries; not always updatable.
- **Materialized View** → Persisted view result; needs REFRESH; faster read but stale until refreshed.

- **Index** → Auxiliary data structure for fast lookup; trades write/space overhead for read speed.
- **Sequence** → Auto-incrementing number generator; used for surrogate keys.
- **Trigger** → Automatic procedure on INSERT/UPDATE/DELETE; use sparingly (hidden logic, hard to debug).
- **Stored Procedure** → Named reusable SQL block; runs on DB server; reduces network round-trips.
- **Function** → Returns a value; can be used in SELECT; deterministic functions can be indexed.
- **Schema (namespace)** → Logical container grouping DB objects; allows same table name in different schemas.
- **Tablespace** → Physical storage location mapping; allows placing tables/indexes on different disks.
- **Partition** → Logical division of a table; range/list/hash; transparent to queries.
- **Extension / Plugin** → Additional capabilities (PostGIS, pgvector, pg_stat_statements).
- **Keywords**: Materialized view = performance at cost of freshness; triggers = power but hidden coupling; stored procedures = server-side logic.

---

6. Relationships & Constraints

Keys

- **Primary Key** → Unique + Not Null; one per table; clustered index by default in MySQL InnoDB.
- **Foreign Key** → References PK of another table; enforces referential integrity; adds join overhead.
- **Unique Constraint** → Allows one null; enforces uniqueness across column(s); backed by unique index.
- **Surrogate Key** → System-generated (auto-increment, UUID); no business meaning; stable.
- **Natural Key** → Business-meaningful (email, SSN); can change; risky as FK target.
- **Composite Key** → Multi-column PK; used in junction tables; column order matters for index use.

Constraints

- **Not Null** → Column must have a value; prevents NULL-related bugs.
- **Check** → Enforce domain rules (`CHECK (age > 0)`); validated on insert/update.
- **Default** → Auto-fill value if none provided; reduces application-level logic.

- **Referential Integrity** → FK relationship must be valid; ON DELETE CASCADE/SET NULL/RESTRICT options.
- **Deferred Constraints** → Validated at end of transaction, not immediately; useful for circular FK scenarios.

Cardinality

- **1:1** → One row in A maps to exactly one row in B; split for optional/large fields.
- **1:N** → One row in A maps to many in B; most common; FK on the "many" side.
- **N:M** → Many-to-many; requires junction/bridge table with two FKs.
- **Keywords**: Always index FK columns; CASCADE deletes can cause surprise mass-deletes; natural keys mutate, surrogate keys don't.

---

7. SQL & Querying

DML — Data Manipulation

- **SELECT** → Read rows; most critical to optimize; drives 90% of performance tuning.
- **INSERT** → Add rows; bulk insert (`INSERT INTO ... VALUES (...),(...)`) far faster than row-by-row.
- **UPDATE** → Modify rows; always use WHERE clause; no WHERE = full table update.
- **DELETE** → Remove rows; no WHERE = full delete; use TRUNCATE for full wipe (faster, not logged per row).
- **MERGE / UPSERT** → Insert or update in one statement; `INSERT ... ON CONFLICT DO UPDATE` (PostgreSQL).

Joins

- **INNER JOIN** → Returns matching rows in both tables; most common.
- **LEFT OUTER JOIN** → All rows from left + matched from right; NULLs where no match.
- **RIGHT OUTER JOIN** → All rows from right + matched from left; rarely needed (rewrite as LEFT JOIN).
- **FULL OUTER JOIN** → All rows from both; NULLs where unmatched.
- **CROSS JOIN** → Cartesian product; every row × every row; use deliberately.
- **SELF JOIN** → Table joined to itself; hierarchy/recursive data (employees and managers).
- **SEMI JOIN** → `WHERE EXISTS (subquery)`; returns rows from left where match exists in right; no duplicates.
- **ANTI JOIN** → `WHERE NOT EXISTS`; rows in left with no match in right.

- **LATERAL JOIN** → Correlated subquery in FROM clause; each row evaluated independently (PostgreSQL).

Advanced SQL

- **Subquery** → Query inside query; correlated subquery re-executes per outer row (expensive).
- **CTE** → `WITH cte AS (...) SELECT...`; readable; often same plan as subquery; not materialized by default.
- **Recursive CTE** → Tree/graph traversal in SQL; anchor + recursive term; use with `UNION ALL`.
- **Window Functions** → `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `LAG()`, `LEAD()`, `SUM() OVER (PARTITION BY...)`.
- **Aggregation** → `SUM`, `AVG`, `COUNT`, `MIN`, `MAX`; `COUNT(*)` counts rows, `COUNT(col)` skips NULLs.
- **GROUP BY / HAVING** → GROUP BY groups rows; HAVING filters groups (WHERE filters rows before grouping).
- **ORDER BY / LIMIT / OFFSET** → Pagination; OFFSET is inefficient at high page numbers (keyset pagination preferred).
- **DISTINCT** → Deduplication; expensive (sort or hash); use sparingly.
- **UNION / INTERSECT / EXCEPT** → Set operations; UNION ALL skips dedup (faster).
- **PIVOT / UNPIVOT** → Rotate rows to columns and back; analytical reporting.
- **JSON operators** → PostgreSQL `->>` (get text), `->` (get JSON), `@>` (contains); index with GIN.
- **Keywords**: Correlated subquery = performance smell; prefer EXISTS over IN with NULLs; keyset pagination over OFFSET; HAVING != WHERE.

---

8. Normalization & Schema Design

Normal Forms

- **1NF** → Atomic values, no repeating groups, each row uniquely identifiable. Eliminate multi-value columns.
- **2NF** → 1NF + no partial dependency (every non-key column depends on full composite key, not part of it).
- **3NF** → 2NF + no transitive dependency (non-key column depending on another non-key column).

- **BCNF** → Stronger 3NF; every determinant must be a candidate key; resolves anomalies 3NF misses.
- **4NF** → BCNF + no multi-valued dependencies; separate independent multi-value facts.
- **5NF** → 4NF + no join dependency; decompose to smallest lossless join components.

Design Strategies

- **Denormalization** → Intentional redundancy for read performance; trade write complexity for query speed.
- **Wide Table** → Many columns; denormalized; analytics-friendly; common in DWH.
- **Entity-Attribute-Value (EAV)** → Flexible schema via rows as attributes; flexible but hard to query and validate; anti-pattern at scale.
- **Polymorphic Association** → One FK pointing to multiple tables via type discriminator; breaks FK integrity.
- **Soft Delete** → `deleted_at TIMESTAMP` instead of DELETE; preserves history; complicates queries (always filter).
- **Audit Columns** → `created_at`, `updated_at`, `created_by`, `updated_by`; add to every table.
- **UUID vs Auto-Increment** → UUID = globally unique, distributed-safe, opaque; auto-increment = sequential, predictable, index-friendly. UUID v7 preserves time-ordering.
- **Surrogate vs Natural Key** → Surrogate = stable, meaningless; Natural = meaningful, can mutate; prefer surrogate for PKs.
- **Keywords**: Normalize to remove anomalies; denormalize for speed; most production schemas are 3NF with selective denormalization.

Interview Deep-Dive Keywords

- "Normalization eliminates update/insert/delete anomalies"
- "BCNF handles cases 3NF doesn't — when a non-trivial determinant isn't a candidate key"
- "Denormalize only after profiling; premature denormalization creates maintenance debt"
- "EAV is a schema-design smell — use JSONB or document DB instead for flexible attributes"

---

9. Indexing

Index Types

- **B-Tree Index** → Default; balanced tree; supports =, <, >, BETWEEN, LIKE 'prefix%'; O(log n) lookup.

- **Hash Index** → O(1) equality; no range support; in-memory or disk (PostgreSQL supports on-disk hash).
- **Bitmap Index** → Bitwise operations; low-cardinality columns (gender, status); great for OLAP (Oracle, Redshift).
- **Composite Index** → Multi-column; column order matters; leftmost prefix rule.
- **Covering Index** → Index includes all columns needed by query; index-only scan; no heap access.
- **Clustered Index** → Data rows physically sorted by index key; one per table; InnoDB PK is clustered.
- **Non-clustered Index** → Separate from data; pointer to heap row; multiple allowed per table.
- **Full-Text Index** → Tokenized inverted index; supports keyword search; `MATCH AGAINST`, `@@` in PostgreSQL.
- **Partial Index** → Index on subset of rows (`WHERE active = true`); smaller, faster for filtered queries.
- **Function-Based Index** → Index on expression (`LOWER(email)`); query must use same expression.
- **GIN** → Generalized Inverted Index; JSON, arrays, full-text in PostgreSQL; fast contains/overlap.
- **GiST** → Generalized Search Tree; geometric/spatial data, full-text; flexible but slower than GIN.
- **BRIN** → Block Range Index; correlates physical storage order to values; tiny size; good for append-only timestamp columns.

Index Strategy

- **Selectivity** → High selectivity (few matching rows) = good index candidate; low selectivity (many matches) = skip.
- **Index Bloat** → Dead index entries after updates/deletes; rebuild or REINDEX periodically.
- **Index Maintenance Cost** → Every write must update all indexes; more indexes = slower writes.
- **Index-Only Scan** → Query resolved entirely from index; no heap access; requires visibility map clean.
- **Leftmost Prefix Rule** → Composite index (a,b,c) supports queries on (a), (a,b), (a,b,c); not (b,c) alone.
- **Keywords**: Don't index every column; index what you query/filter/sort/join on; monitor unused indexes with `pg_stat_user_indexes`.

Interview Deep-Dive Keywords

- "B-tree for general purpose; Hash for equality only; Bitmap for low-cardinality OLAP; GIN for JSON/arrays"
- "Composite index column order: highest selectivity first OR match query WHERE clause order"
- "Covering index eliminates heap fetch — biggest single-query win available"
- "Partial index is underused: `WHERE deleted_at IS NULL` index is smaller and faster than full-column index"

---

10. Query Optimization

How the Optimizer Works

- **Query Plan / Execution Plan** → Steps the DB will take; generated by optimizer; inspect with EXPLAIN.
- **Cost-Based Optimizer (CBO)** → Estimates cost using table statistics; chooses lowest-cost plan.
- **Statistics** → Row counts, column cardinality, histograms; stale stats = bad plans; run ANALYZE/UPDATE STATISTICS.
- **EXPLAIN** → Shows plan without executing; estimates only.
- **EXPLAIN ANALYZE** → Executes query and shows actual vs estimated rows; key diagnostic tool.
- **Cardinality Estimation** → Optimizer's guess at rows per step; wrong estimation = wrong join order = slow query.
- **Selectivity** → Fraction of rows a predicate matches; drives index vs scan decision.

Plan Nodes

- **Index Scan** → Uses index to find rows; fast for selective queries.
- **Index-Only Scan** → Index covers all needed columns; no heap access; fastest.
- **Bitmap Index Scan** → Builds bitmap of matching pages; efficient for multiple conditions.
- **Sequential Scan (Full Table Scan)** → Reads every page; acceptable for small tables or low-selectivity queries; expensive on large tables.
- **Nested Loop Join** → Outer row × index lookup on inner; good when inner result is small.
- **Hash Join** → Build hash table on smaller side; probe with larger; good for large unsorted joins.
- **Merge Join** → Both sides sorted; efficient when inputs already sorted; can use indexes.

- **Predicate Pushdown** → Filter applied close to data source; reduces rows early in plan.
- **Parallel Query** → Multiple workers scan/aggregate in parallel; controlled by `max_parallel_workers_per_gather`.

Optimization Techniques

- **Query Rewrite** → Restructure SQL for better plan (replace correlated subquery with JOIN, etc.).
- **Query Hints** → Force index/join type; use as last resort (bypasses optimizer knowledge).
- **Materialized CTE** → PostgreSQL 12+ CTEs not always materialized; use `MATERIALIZED` keyword when needed.
- **Pagination** → Keyset (`WHERE id > last_seen_id LIMIT 20`) over OFFSET; OFFSET reads and discards all prior rows.
- **Keywords**: Always EXPLAIN ANALYZE before and after optimization; watch for "rows=1 actual rows=100000" — that's stale stats.

Interview Deep-Dive Keywords

- "Sequential scan isn't always wrong — for tables < ~10% selectivity it beats index scan"
- "Hash join for large unordered joins; nested loop for small inner sets with index"
- "Stale statistics are the #1 cause of bad query plans — ANALYZE after bulk loads"
- "OFFSET pagination is O(n) — keyset pagination is O(1) per page"

---

11. Transactions

ACID Properties

- **Atomicity** → All operations in a transaction succeed or all are rolled back. No partial updates.
- **Consistency** → Transaction brings DB from one valid state to another; constraints must hold.
- **Isolation** → Concurrent transactions don't interfere; degree controlled by isolation level.
- **Durability** → Committed transaction persists even after crash; guaranteed by WAL/redo log.

Isolation Levels & Anomalies

Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	✅ Possible	✅ Possible	✅ Possible

Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Committed	❌ Prevented	✅ Possible	✅ Possible
Repeatable Read	❌ Prevented	❌ Prevented	✅ Possible (MySQL prevents)
Serializable	❌ Prevented	❌ Prevented	❌ Prevented

- **Dirty Read** → Reading uncommitted data from another transaction.
- **Non-Repeatable Read** → Same row read twice in a transaction returns different values.
- **Phantom Read** → Re-running a range query returns different set of rows.
- **Commit** → Permanently persist all changes; releases locks.
- **Rollback** → Undo all changes in current transaction.
- **Savepoint** → Partial rollback point within a transaction; `SAVEPOINT sp1; ROLLBACK TO sp1`.
- **Autocommit** → Each statement is its own transaction by default; disable for multi-statement transactions.
- **Long-Running Transactions** → Hold locks; block others; bloat MVCC; kill on detection.
- **Transaction Log** → Sequential write of changes; basis for durability + replication.
- **Keywords**: Read Committed = PostgreSQL default; Repeatable Read = MySQL default; Serializable = safest but slowest.

Interview Deep-Dive Keywords

- "ACID is not free — higher isolation = more locking = lower concurrency"
- "Read Committed prevents dirty reads but allows phantom reads — fine for most OLTP"
- "Long-running transactions are the enemy of MVCC — they bloat the transaction ID space and dead tuples"
- "Savepoints allow nested transaction-like behavior without full rollback"

---

12. Concurrency Control

Locking

- **Row-Level Lock** → Finest granularity; high concurrency; most RDBMS default for DML.
- **Table-Level Lock** → Entire table locked; low concurrency; used by DDL (ALTER TABLE).
- **Page-Level Lock** → Lock a storage page; compromise between row and table.
- **Shared Lock (S)** → Multiple readers can hold simultaneously; blocks exclusive locks.
- **Exclusive Lock (X)** → Only one holder; blocks all other locks; held during writes.

- **Intention Lock** → Hierarchy signal (IX, IS); allows table-level check without row scan.
- **Deadlock** → T1 waits for T2; T2 waits for T1; DB detects and kills one (victim selection).
- **Livelock** → Both transactions keep retrying, neither progresses; rare in DBs.
- **Lock Escalation** → Row locks promoted to table lock under pressure; reduces overhead but blocks reads.
- **Advisory Lock** → Application-managed locks (PostgreSQL `pg_advisory_lock`); not tied to data.

MVCC

- **MVCC (Multi-Version Concurrency Control)** → Readers don't block writers; each transaction sees a snapshot.
- **Snapshot Isolation** → Transaction reads from a consistent point-in-time snapshot; no dirty reads.
- **Tuple Versioning** → Old row versions kept until no active transaction needs them.
- **Dead Tuples** → Outdated row versions no longer needed; VACUUM reclaims space.
- **Transaction ID (XID)** → Monotonically increasing; every transaction assigned one; wraps after ~2 billion (freeze mechanism prevents issues).

Optimistic vs Pessimistic

- **Pessimistic Locking** → Lock before access; safe; lower concurrency; `SELECT ... FOR UPDATE`.
- **Optimistic Locking** → Assume no conflict; validate at commit via version column; retry on conflict.
- **Version Column** → `updated_at` or `version INT`; compare on UPDATE to detect concurrent modification.
- **CAS (Compare-And-Swap)** → `UPDATE ... WHERE version = $expected`; atomic optimistic update.
- **Keywords**: MVCC enables high read concurrency; VACUUM is mandatory maintenance for MVCC; use `SELECT FOR UPDATE` to lock specific rows.

Interview Deep-Dive Keywords

- "MVCC = readers never block writers; writers never block readers — PostgreSQL's superpower"
- "Deadlock prevention: always acquire locks in the same order across transactions"
- "Optimistic locking works well when conflicts are rare; pessimistic when contention is high"

- "Long-running transactions bloat dead tuples — autovacuum can't reclaim space with an old snapshot open"

---

13. Consistency Models

Spectrum

- **Strong Consistency** → Every read sees the most recent write; linearizable; highest cost.
- **Linearizability** → Operations appear instantaneous; global order consistent with real time.
- **Sequential Consistency** → Operations in program order per process; global order exists but not real-time.
- **Causal Consistency** → Causally related operations seen in order; unrelated can be reordered.
- **Read-Your-Writes** → After a write, the same client always reads its own write.
- **Monotonic Reads** → Once a value is read, older values are never returned in subsequent reads.
- **Session Consistency** → Read-your-writes + monotonic reads within a session.
- **Eventual Consistency** → Writes propagate eventually; replicas converge; no ordering guarantee.
- **Bounded Staleness** → Reads may lag by at most K versions or T seconds (Cosmos DB).

Theorems

- **CAP Theorem** → Distributed system can only guarantee 2 of 3: Consistency, Availability, Partition Tolerance. Partition is unavoidable → choose CP or AP.
- **PACELC Theorem** → Extends CAP: when no partition (E), trade Latency vs Consistency. Better model for always-on systems.
- **CP Systems** → Consistent + Partition-tolerant; may reject requests during partition (HBase, Zookeeper, etcd).
- **AP Systems** → Available + Partition-tolerant; may return stale data (Cassandra, DynamoDB, CouchDB).
- **Keywords**: CAP is binary; PACELC is a spectrum; most systems let you tune consistency per-operation.

Interview Deep-Dive Keywords

- "CAP says pick 2, but partition tolerance is non-negotiable in distributed systems — really pick CP or AP"

- "Eventual consistency is a spectrum — 'how eventual' depends on replication lag and topology"
- "Cassandra lets you tune per-query: ONE (AP) vs QUORUM (closer to CP)"
- "Strong consistency has latency cost — cross-datacenter synchronous replication adds 100ms+ RTT"

---

14. Storage Engine & Architecture

Storage Layouts

- **Row Store** → Row data stored together; good for OLTP (full row fetch, updates in place).
- **Column Store** → Column data stored together; good for OLAP (scan single column across all rows, compresses well).
- **Heap File** → Unsorted rows; pages; most row-store DBs use this as base structure.
- **Page / Block** → Fixed-size unit (usually 8KB in PostgreSQL, 16KB in MySQL); unit of I/O.
- **Tuple / Slot** → Row within a page; slot array tracks tuple offsets within a page.
- **Buffer Pool / Buffer Cache** → RAM cache of recently accessed pages; goal: serve reads from memory.
- **LRU / Clock-Sweep Eviction** → Pages evicted when pool is full; PostgreSQL uses clock-sweep (cheaper than LRU).

Write Path

- **Write-Ahead Log (WAL)** → Changes written to log before data pages; ensures durability; enables replication.
- **LSM Tree (Log-Structured Merge Tree)** → Append-only writes; sequential I/O; background compaction (RocksDB, Cassandra, LevelDB).
- **SSTable** → Sorted String Table; immutable sorted file; component of LSM.
- **Memtable** → In-memory write buffer in LSM; flushed to SSTable on fill.
- **Compaction** → Merging SSTables to remove dead data; reduces read amplification.
- **Bloom Filter** → Probabilistic structure; checks if key might exist in SSTable; avoids unnecessary disk reads.

Write Amplification Trade-offs

- **Write Amplification** → Each logical write causes multiple physical writes; LSM has high WA during compaction.
- **Read Amplification** → Each read requires checking multiple SSTables; mitigated by bloom filters + compaction.

- **Space Amplification** → Old versions occupy space until compaction; LSM needs ~1.1x–2x space.
- **Compression** → LZ4, Snappy, Zstandard; column stores compress much better than row stores.
- **Keywords**: LSM = write-optimized (RocksDB, Cassandra); B-tree = read-optimized (PostgreSQL, MySQL); trade-off is write vs read amplification.

---

15. Database Internals (Deep Dive)

WAL & Recovery

- **WAL Segment** → Fixed-size log file (default 16MB in PostgreSQL); rotated as filled.
- **Checkpoint** → Flush dirty pages to disk + WAL position marker; limits crash recovery time.
- **Log Sequence Number (LSN)** → Monotonically increasing position in WAL; tracks replication progress.
- **Redo Log** → Replay forward from checkpoint to re-apply committed changes after crash.
- **Undo Log** → Roll back uncommitted changes; used by InnoDB for MVCC and rollback.
- **ARIES Algorithm** → Analysis + Redo + Undo; standard WAL-based recovery algorithm (Steal/No-Force model).
- **Steal Policy** → Dirty pages can be flushed before transaction commits (requires undo log).
- **No-Force Policy** → Committed pages don't have to be immediately flushed to disk (requires redo log).
- **Crash Recovery** → DB replays WAL from last checkpoint on restart; guarantees durability.

MVCC Internals

- **Tuple Versioning** → Each row has `xmin` (created by) and `xmax` (deleted by) transaction IDs.
- **Visibility Rules** → `xmin` committed + `xmax` not committed = visible to current transaction.
- **Dead Tuples** → Rows with `xmax` committed; invisible but still occupy space until VACUUM.
- **VACUUM** → Reclaims dead tuple space; does not shrink table file (VACUUM FULL does, with lock).
- **Autovacuum** → Background process; triggered by dead tuple threshold; critical to maintain.
- **Visibility Map** → Tracks pages with only live tuples; enables index-only scans and speeds VACUUM.
- **Transaction ID Wraparound** → XID is 32-bit; wraps after ~2B; FREEZE mechanism prevents data loss.

- **Freeze** → Mark old tuples as permanently visible; sets `xmin = FrozenTransactionId`.

Buffer Pool Internals

- **Dirty Page** → Modified page in buffer pool not yet flushed to disk.
- **Page Latch** → Short-duration in-memory lock protecting a buffer pool page (not the same as a database lock).
- **Double-Write Buffer** → InnoDB: writes pages to double-write buffer first; protects against torn pages.
- **Free Space Map** → Tracks available space in heap pages; used by INSERT to find space.
- **Bloat & Fragmentation** → Frequent updates/deletes leave gaps; causes I/O inefficiency; VACUUM + CLUSTER help.
- **Keywords**: LSN is the backbone of replication; checkpoint frequency controls crash recovery time vs I/O cost; autovacuum tuning is critical for MVCC health.

Interview Deep-Dive Keywords

- "WAL comes before data — that's what makes crash recovery possible"
- "MVCC dead tuples accumulate when autovacuum can't keep up or long transactions hold back the horizon"
- "Checkpoint too infrequent = long crash recovery; too frequent = I/O spike"
- "Transaction ID wraparound can corrupt data — monitor age(datfrozenxid) and vacuum accordingly"

---

16. Data Distribution & Partitioning

Partitioning

- **Horizontal Partitioning** → Split rows across partitions; same schema per partition.
- **Vertical Partitioning** → Split columns into separate tables; useful for hot vs cold columns.
- **Range Partitioning** → Partition by value range (`created_at >= '2024-01-01'`); natural for time-series.
- **Hash Partitioning** → Partition by hash of key; even distribution; good for even write spread.
- **List Partitioning** → Explicit value lists per partition (`country IN ('US','CA')`).
- **Partition Pruning** → Optimizer skips non-matching partitions; dramatically reduces scan cost.
- **Partition Key Selection** → Choose based on query patterns; always include in WHERE for pruning.

- **Global Index vs Local Index** → Global spans all partitions (complex maintenance); local is per-partition (preferred).
- **Hot Partition** → One partition receiving disproportionate writes; common with sequential keys + hash sharding.

Sharding

- **Sharding** → Distribute data across separate DB instances (not just table partitions); horizontal scaling.
- **Shard Key** → Determines which shard holds a row; most important design decision.
- **Range Sharding** → Contiguous key ranges per shard; hot spots on sequential keys.
- **Hash Sharding** → Even distribution but loses range query locality.
- **Consistent Hashing** → Virtual nodes on a ring; node add/remove minimizes data movement.
- **Resharding** → Moving data as shards become unbalanced; expensive and complex.
- **Cross-Shard Queries** → Require scatter-gather; expensive; avoid by denormalization or co-location.
- **Hotspot** → One shard overloaded; add random suffix to shard key to spread writes.
- **Data Locality** → Related data on same shard; reduces cross-shard joins.
- **Keywords**: Shard key = most critical sharding decision; wrong key = hotspot or cross-shard joins; you can't change shard key easily.

Interview Deep-Dive Keywords

- "Partition key and shard key must reflect query access patterns — not just even distribution"
- "Consistent hashing minimizes resharding cost when nodes are added/removed"
- "Hash sharding spreads write load but destroys range query locality"
- "Cross-shard transactions require distributed transaction protocols (2PC or Saga) — avoid if possible"

---

17. Replication

Replication Types

- **Synchronous Replication** → Primary waits for replica to confirm write; zero data loss; higher latency.
- **Asynchronous Replication** → Primary doesn't wait; lower latency; potential data loss on failover.

- **Semi-Synchronous** → Wait for at least one replica to confirm; MySQL option.
- **Leader-Follower (Master-Slave)** → Single write node (leader); multiple read replicas (followers); common pattern.
- **Multi-Master** → Multiple nodes accept writes; conflict resolution needed; complex.
- **Active-Active** → Both nodes serving reads and writes; highest availability; hardest consistency.
- **Active-Passive** → One active, one passive standby; simpler; some downtime on failover.

Replication Mechanics

- **Log-Based Replication** → Ship WAL/binlog to replica; most reliable; used by PostgreSQL streaming replication, MySQL binlog.
- **Logical Replication** → Replicate logical changes (row-level); allows cross-version, cross-DB replication.
- **Physical Replication** → Replicate exact storage blocks; same DB version required; faster for standbys.
- **Replication Lag** → Delay between primary write and replica applying it; monitor constantly.
- **Replica Reads** → Route read queries to replicas; reduces primary load; may return stale data.
- **Failover** → Promote a replica to leader on primary failure; automated (Patroni, RDS) or manual.
- **Read Endpoint / Write Endpoint** → Route writes to primary, reads to replicas via proxy.
- **Replication Slot** → PostgreSQL mechanism to track replica WAL position; ensures WAL not deleted before replica consumes; risk of WAL accumulation if replica falls behind.
- **Keywords**: Monitor replication lag; never let replication slots accumulate WAL indefinitely (disk fill risk); logical replication for zero-downtime major version upgrades.

---

18. Distributed Databases

Distributed Transactions

- **Distributed Transaction** → Single transaction spanning multiple nodes/shards; requires coordination.
- **Two-Phase Commit (2PC)** → Phase 1: prepare (all vote yes/no); Phase 2: commit or abort. Blocking if coordinator fails.
- **Three-Phase Commit (3PC)** → Adds pre-commit phase to avoid blocking; more network rounds; rarely used.

- **Saga Pattern** → Sequence of local transactions; compensating transactions on failure; no distributed lock.
- **Choreography Saga** → Services react to events; no central coordinator; decoupled.
- **Orchestration Saga** → Central orchestrator drives saga steps; easier to trace; single point of control.

Consensus & Coordination

- **Raft** → Leader-based consensus; easier to understand than Paxos; used by etcd, CockroachDB, TiKV.
- **Paxos** → Original consensus algorithm; complex; basis for Spanner, Chubby.
- **Leader Election** → Consensus to elect single writer; prevents split-brain.
- **Quorum** → Majority of nodes must agree for read/write to succeed (e.g. W + R > N).
- **Split Brain** → Two nodes both believe they are leader; prevented by quorum and fencing.
- **Fencing Token** → Monotonically increasing token from lock service; old leader's writes rejected.
- **Epoch / Term Number** → Logical time period of a leader; used to detect stale leaders.
- **Gossip Protocol** → Peer-to-peer state dissemination; nodes share info with random neighbors; eventual convergence.

Interview Deep-Dive Keywords

- "2PC is blocking — coordinator crash leaves participants in doubt; avoid long-running 2PC"
- "Saga replaces distributed transactions with compensating transactions — eventual consistency"
- "Quorum W=2, R=2, N=3 guarantees at least one node overlap — strong consistency"
- "Split brain is prevented by quorum; fencing tokens ensure old leaders don't write after demotion"

---

19. Performance & Scaling

Connection Management

- **Connection Pooling** → Reuse connections; reduce connection overhead; PgBouncer (PostgreSQL), ProxySQL (MySQL).
- **Pool Modes** → Session (connection per client), Transaction (connection per transaction — more efficient), Statement.
- **Max Connections** → DB has a hard limit; each connection uses memory (~5MB in PostgreSQL); pool to stay under limit.

- **Idle Connections** → Consume memory; set pool idle timeout; close unused connections.
- **Prepared Statements** → Parse once, execute many times; cache execution plan; reduce parse overhead.

Caching

- **Buffer Pool Hit Ratio** → % of reads served from memory; target >99% for OLTP; tune `shared_buffers`.
- **Application-Level Cache** → Redis/Memcached; cache query results; reduces DB load; cache invalidation is hard.
- **Read-Through Cache** → Cache checks DB on miss; transparent to app.
- **Write-Through Cache** → Write to cache + DB simultaneously; consistent but slower writes.
- **Cache Invalidation** → TTL-based or event-based; stale cache is a consistency risk.

Scaling Patterns

- **Vertical Scaling** → Bigger machine (more RAM/CPU/disk); limited ceiling; no code change needed.
- **Horizontal Scaling** → Add more nodes; requires sharding/replication; more complex.
- **Read Replicas** → Offload read traffic; eventual consistency risk; session affinity for read-your-writes.
- **Write Scaling** → Shard; partition; CQRS; harder than read scaling.
- **Batch Processing** → Group writes into batches; dramatically reduces transaction overhead.
- **Bulk Insert** → `COPY` (PostgreSQL) / `LOAD DATA INFILE` (MySQL); 10–100x faster than row-by-row INSERT.
- **Query Parallelism** → Multiple CPU cores for single query; tune `max_parallel_workers_per_gather`.
- **Statement Timeout** → Kill long-running queries automatically; prevents runaway queries from starving others.

Interview Deep-Dive Keywords

- "PgBouncer transaction mode can multiplex 1000 app connections to 50 DB connections"
- "Read replica scaling is easy; write scaling requires sharding — avoid until necessary"
- "COPY is the fastest way to load data; use staging tables + COPY for bulk loads"
- "Statement timeouts are essential in production — without them, one bad query can lock the system"

---

20. NoSQL Systems

Core Concepts

- **BASE** → Basically Available, Soft state, Eventually consistent; NoSQL alternative to ACID.
- **Schema-less** → No enforced schema at DB level; application owns schema; flexible but risky at scale.
- **Denormalization** → Embed related data; optimize for read access pattern; write duplication accepted.
- **Document Embedding vs Referencing** → Embed for data always accessed together; reference when data shared across documents.

Key-Value Stores

- **Redis** → In-memory; strings/hashes/lists/sets/sorted sets; pub/sub; Lua scripts; persistence via RDB/AOF.
- **DynamoDB** → Managed KV + document; single-digit ms latency; partition key + sort key; global tables.
- **Memcached** → Pure cache; no persistence; simple; multi-threaded; no complex data structures.
- **TTL** → Auto-expire keys; critical for session stores and caches.

Document Stores

- **MongoDB** → BSON documents; flexible schema; secondary indexes; aggregation pipeline; transactions (v4+).
- **Collection Design** → Embed for 1:few; reference for 1:many (large); never grow arrays unboundedly.
- **CouchDB** → HTTP API; eventual consistency; MVCC; conflict resolution at application level.

Wide-Column Stores

- **Cassandra** → Leaderless; partition key + clustering key; tunable consistency; write-optimized LSM; no joins.
- **Partition Key Design** → High cardinality + even distribution; hot partition = death knell.
- **Sort Key / Clustering Key** → Controls ordering within a partition; models range queries.
- **Consistency Tuning** → ONE (fast, AP), QUORUM (balanced), ALL (slow, CP); per-query setting.
- **HBase** → Hadoop-native; sorted column-family; strong consistency; ZooKeeper coordination.

Interview Deep-Dive Keywords

- "NoSQL is not always the answer — use when schema flexibility, scale, or access pattern demands it"
- "Cassandra partition key determines node; clustering key determines order within partition"
- "MongoDB embedding vs referencing: embed when data is always read together and doesn't exceed 16MB doc limit"
- "DynamoDB single-table design: pack multiple entity types into one table using pk/sk patterns"

---

21. Graph Databases

Core Concepts

- **Node** → Entity; has labels (type) and properties (attributes).
- **Edge / Relationship** → Connection between nodes; has type and direction; can have properties.
- **Property Graph Model** → Nodes + edges both carry properties; most common model (Neo4j).
- **RDF (Resource Description Framework)** → Triple store (subject-predicate-object); semantic web; SPARQL query language.
- **Adjacency List** → Standard storage representation; graph DBs store this natively for O(1) relationship traversal.

Query & Traversal

- **Cypher** → Neo4j's declarative graph query language; `MATCH (n:Person)-[:KNOWS]->(m)`.
- **Gremlin** → Apache TinkerPop; imperative traversal language; multi-vendor support.
- **SPARQL** → RDF graph query language; semantic/knowledge graphs.
- **BFS / DFS Traversal** → Shortest path uses BFS; DFS for exhaustive graph exploration.
- **Shortest Path** → `shortestPath()` in Cypher; Dijkstra/Bellman-Ford algorithms.
- **Path Query** → Find paths matching a pattern; variable-length relationships `[:KNOWS*1..3]`.

Graph Algorithms

- **PageRank** → Node importance based on incoming link weight; social network influence.

- **Betweenness Centrality** → Nodes on most shortest paths; network bottleneck detection.
- **Community Detection** → Louvain / Label Propagation; cluster discovery.
- **Knowledge Graph** → Structured representation of entities and their relationships; used in search, recommendation.

Databases

- **Neo4j** → Dominant native graph DB; Cypher; ACID; clustering.
- **Amazon Neptune** → Managed; supports both Property Graph (Gremlin) and RDF (SPARQL).
- **TigerGraph** → High-performance; GSQL; parallel traversal; enterprise scale.
- **Keywords**: Use graph DB when relationships are the primary query focus; relational recursive CTEs work for simple hierarchies but degrade at depth.

---

22. Time-Series Databases

Core Concepts

- **Time Index** → Timestamp as primary dimension; queries always filter by time range.
- **Write Pattern** → Append-heavy; rows rarely updated; delete by retention policy.
- **Cardinality Problem** → High-cardinality tags (user_id as label) explode index size; leading killer in Prometheus.
- **Downsampling** → Reduce resolution over time (1s → 1m → 1h); trade precision for storage.
- **Retention Policy** → Auto-delete data older than N days; critical for storage management.
- **Compression** → Delta encoding (timestamps), XOR encoding (floats); 10x+ compression common.

Databases

- **InfluxDB** → Leading TSDB; line protocol; Flux query language; built-in downsampling; retention policies.
- **TimescaleDB** → PostgreSQL extension; `time_bucket()` aggregation; hypertables; continuous aggregates.
- **Prometheus** → Pull-based metrics store; PromQL; local storage; not designed for long-term retention.
- **Druid** → Real-time + historical; sub-second OLAP; pre-aggregation; Kafka ingestion.
- **OpenTSDB** → HBase-backed; scalable; older; being replaced by newer TSDBs.

Patterns

- **Hypertable** → TimescaleDB abstraction; automatically partitions by time; transparent to SQL.
- **Continuous Aggregates** → Pre-computed rollups refreshed incrementally; materialized time buckets.
- **Out-of-Order Data** → Late arriving data; buffering window needed; hard for streaming ingestion.
- **Last-Value Cache** → Serve latest reading without full table scan; common in IoT dashboards.
- **Keywords**: Time-series workloads = append-only + time-range queries + aggregation; never use general RDBMS for high-cardinality metrics.

---

23. Vector Databases & AI/ML Data

Core Concepts

- **Vector Embedding** → Dense numerical representation of data (text, image, audio) in high-dimensional space; produced by ML models.
- **Similarity Search** → Find vectors closest to a query vector; basis of semantic search, recommendations, RAG.
- **Distance Metrics** → Cosine similarity (angle between vectors, normalized); Euclidean (L2 distance); Dot product (unnormalized cosine); choose based on embedding model training.
- **ANN (Approximate Nearest Neighbor)** → Trade perfect recall for speed; necessary at scale (exact search = O(n) per query).

Index Algorithms

- **HNSW (Hierarchical Navigable Small World)** → Graph-based; fast query; high recall; high memory footprint; most popular.
- **IVF (Inverted File Index)** → Cluster vectors into centroids; search only nearest clusters; fast build; tunable recall vs speed.
- **FAISS** → Facebook AI Similarity Search; library (not a DB); supports IVF, HNSW, PQ; used under the hood by many vector DBs.
- **Product Quantization (PQ)** → Compress vectors; huge memory savings; some recall loss.
- **Recall vs Latency** → HNSW parameters (`ef`, `M`) trade recall % for query latency; tune per use case.

Vector Databases

- **pgvector** → PostgreSQL extension; `vector` type; HNSW + IVF indexes; SQL-native; easy integration.
- **Pinecone** → Managed; serverless; namespaces; metadata filtering; production-ready.
- **Weaviate** → Open-source + managed; multi-modal; built-in vectorizer modules; GraphQL API.
- **Qdrant** → Rust-based; fast; filtering on payload; sparse + dense vector support.
- **Milvus** → Open-source; enterprise-scale; disaggregated storage/compute.
- **Chroma** → Lightweight; Python-native; dev/prototype focus.

Patterns

- **RAG (Retrieval-Augmented Generation)** → Embed query → retrieve top-K similar chunks → pass to LLM as context; reduces hallucination.
- **Chunking Strategy** → Split documents into chunks before embedding; chunk size affects recall quality.
- **Metadata Filtering** → Filter by structured attributes alongside vector similarity (`language='en' AND type='article'`).
- **Hybrid Search** → Combine dense vector search with sparse keyword search (BM25); better recall for named entities.
- **Namespace / Collection** → Isolate vector spaces per tenant/use case; multi-tenancy pattern.
- **Index Build Time** → HNSW build is slower than IVF; plan for rebuild on large dataset updates.
- **Keywords**: Vector DBs are append-optimized; updates require delete + re-insert; choose index type based on dataset size and recall requirements.

Interview Deep-Dive Keywords

- "pgvector is enough for <10M vectors; purpose-built vector DB for production scale"
- "HNSW = high recall, high memory; IVF = tunable, lower memory but needs training step"
- "RAG quality depends on chunking strategy as much as retrieval — overlap chunks to preserve context"
- "Hybrid search (vector + BM25) beats pure vector search on benchmarks for most real-world queries"

---

24. Data Warehousing & Analytics

Architecture

- **Data Warehouse** → Central analytical repository; historical; read-optimized; structured (Redshift, Snowflake, BigQuery).
- **Data Lake** → Raw data at any scale and format; schema-on-read; cheap storage (S3, GCS, ADLS).
- **Data Mart** → Subset of DWH for a specific department/function; faster, more focused queries.
- **Lakehouse** → Combines data lake storage with warehouse query capabilities (Delta Lake, Apache Iceberg, Hudi).
- **ETL** → Extract → Transform → Load; transform before loading; traditional DWH approach.
- **ELT** → Extract → Load → Transform; load raw then transform in DWH; modern cloud approach.

Schema Patterns

- **Star Schema** → Central fact table surrounded by dimension tables; simple joins; fast aggregations.
- **Snowflake Schema** → Normalized dimensions (dimensions have sub-dimensions); less redundancy; more joins.
- **Fact Table** → Records events/transactions; has FKs to dimensions + numeric measures; large.
- **Dimension Table** → Descriptive attributes (customer, product, date); smaller; denormalized.
- **SCD Type 1** → Overwrite dimension value; no history kept.
- **SCD Type 2** → New row per change + effective dates; full history; most common.
- **SCD Type 3** → Previous + current column; limited history.
- **Conformed Dimension** → Shared dimension across multiple fact tables/data marts; enables drill-across.

Technology

- **Columnar Storage** → Parquet, ORC; compress per-column; skip non-queried columns; standard for analytics.
- **MPP (Massively Parallel Processing)** → Distribute query across many nodes; Redshift, Snowflake, BigQuery.
- **Query Pushdown** → Push filters to storage layer before transferring data; critical for Parquet/cloud DWH performance.

- **Partition Elimination** → Skip irrelevant partitions based on query predicates; requires partition key in WHERE.
- **dbt (Data Build Tool)** → SQL-based transformation layer; version-controlled models; lineage tracking.
- **Keywords**: Star schema = fast analytics, some redundancy; Snowflake schema = normalized, more joins; always use SCD Type 2 when history matters.

---

25. Streaming & Real-Time Data

Core Concepts

- **Event Streaming** → Continuous flow of events; ordered log; Kafka, Kinesis, Pulsar.
- **Change Data Capture (CDC)** → Capture every row-level change from DB WAL/binlog; stream downstream.
- **Debezium** → CDC connector framework; Kafka-native; supports PostgreSQL, MySQL, MongoDB, Oracle.
- **Outbox Pattern** → Write to `outbox` table atomically with business transaction; CDC picks up and publishes; guarantees at-least-once delivery.
- **Transactional Outbox** → Solve dual-write problem; DB insert + message publish atomically.
- **At-Least-Once Delivery** → Message may be delivered multiple times; consumer must be idempotent.
- **Exactly-Once Semantics** → Hardest to achieve; Kafka transactions + idempotent producers.

Stream Processing

- **Windowing** → Group events into time windows for aggregation.
  - **Tumbling Window** → Fixed non-overlapping windows (0–60s, 60–120s).
  - **Sliding Window** → Overlapping windows; every N seconds advance by M.
  - **Session Window** → Activity-based; gap triggers new window.
- **Watermark** → Heuristic for out-of-order event handling; defines how late data is tolerated.
- **Late Arriving Data** → Events arriving after watermark; dropped or handled by late data trigger.
- **CQRS** → Command Query Responsibility Segregation; separate write model from read model; stream syncs them.
- **Event Sourcing** → Store state as sequence of events; replay to reconstruct state; append-only log.

- **Keywords**: Outbox pattern is the gold standard for DB-to-stream consistency; idempotency is mandatory for at-least-once consumers.

---

26. Schema Migration & Evolution

Migration Tools

- **Flyway** → SQL-first migrations; versioned (`V1__init.sql`) + repeatable (`R__view.sql`); checksums.
- **Liquibase** → XML/YAML/SQL changesets; more complex than Flyway; database-agnostic diff.
- **Alembic** → Python/SQLAlchemy migration tool; `upgrade` + `downgrade` functions.
- **Baseline Migration** → Mark existing schema as starting point for migration tool.

Safe Migration Patterns

- **Additive Changes (Safe)** → Add column/table/index; non-breaking; always backward compatible.
- **Destructive Changes (Risky)** → Drop column/table; rename; type change; always verify zero readers first.
- **Expand-Contract Pattern (Parallel Change)** → Add new → backfill → switch code → drop old; zero-downtime rename/type-change.
- **Column Rename Strategy** → Add new column → dual-write both → backfill old → switch reads → drop old.
- **Zero-Downtime Migration** → Schema changes that don't lock the table or break running queries.
- **Online DDL** → `ALTER TABLE` without locking; MySQL pt-online-schema-change, gh-ost; PostgreSQL locks less aggressively.
- **gh-ost (GitHub's Online Schema Migrator)** → Ghost table approach; uses binlog; no triggers; MySQL.
- **pt-online-schema-change** → Percona tool; trigger-based; older MySQL approach.

Deployment Patterns

- **Backward-Compatible Schema** → New code + old code can run simultaneously; required for rolling deploys.
- **Blue-Green Database Deployment** → Two identical DB environments; cut over at deploy; works for small DBs.
- **Versioned Migrations** → Every change is a versioned file; never edit; only append.

- **Rollback Strategy** → Plan compensation SQL for every migration; test rollback in staging.
- **Keywords**: Never skip the expand-contract pattern for renames; always test migrations on a production-sized dataset.

Interview Deep-Dive Keywords

- "Adding a NOT NULL column to a large table acquires AccessExclusiveLock — use DEFAULT + backfill instead"
- "Expand-contract is mandatory for zero-downtime renames in systems with rolling deploys"
- "gh-ost is safer than pt-osc for MySQL — no triggers means less replication lag impact"
- "Always run migrations in a transaction where supported — partial migration = corrupted schema"

---

27. Multi-Model & Polyglot Persistence

Core Patterns

- **Polyglot Persistence** → Use different DB types for different services based on data access patterns.
- **Multi-Model Database** → Single DB supporting multiple models (ArangoDB: KV + document + graph; Cosmos DB: multiple APIs).
- **Database per Service** → Microservices own their data; no shared schema; service boundary = DB boundary.
- **Shared Database Anti-Pattern** → Multiple services on one DB; tight coupling; deployment dependency.

Synchronization Challenges

- **Dual Write Problem** → Writing to DB and messaging system separately; one can fail; use Outbox pattern instead.
- **CDC-Based Sync** → Use change data capture to propagate DB changes to secondary stores (Elasticsearch, Redis).
- **Application-Level Sync** → Write to both stores in application code; complex, error-prone.
- **Consistency Across Stores** → Impossible to have strong consistency across heterogeneous stores; design for eventual.
- **Data Ownership** → Each store owned by one service; others query via API; prevents coupling.

Common Combinations

- **RDBMS + Cache** → PostgreSQL + Redis; cache hot reads; invalidate on write.
- **RDBMS + Search** → PostgreSQL + Elasticsearch; sync via CDC/Debezium; full-text search offloaded.
- **RDBMS + Graph** → PostgreSQL + Neo4j; relational for transactions; graph for relationship queries.
- **Event Store + Read Model** → Event sourcing store + denormalized read model; CQRS pattern.
- **Keywords**: Polyglot persistence solves access pattern mismatch; operational complexity is the cost; CDC makes sync reliable.

---

28. Backup & Recovery

Backup Types

- **Full Backup** → Complete copy of all data; slowest; largest; needed as recovery baseline.
- **Incremental Backup** → Only changes since last backup; faster; must chain to full for restore.
- **Differential Backup** → Changes since last full; faster restore than incremental; larger than incremental.
- **Logical Backup** → SQL dump (`pg_dump`, `mysqldump`); portable; slow for large DBs; version-agnostic.
- **Physical Backup** → Copy of data files (`pg_basebackup`, `xtrabackup`); fast; version-dependent.
- **WAL Archiving** → Archive WAL segments continuously; enables point-in-time recovery.
- **Snapshots** → Storage-level (EBS snapshot, filesystem snapshot); near-instant; consistent if DB quiesced.

Recovery

- **Restore** → Recreate DB from backup; test regularly (untested backup = no backup).
- **Point-in-Time Recovery (PITR)** → Restore to any moment using base backup + WAL archive; minimum RPO.
- **RTO (Recovery Time Objective)** → Max acceptable downtime; drives HA architecture decisions.
- **RPO (Recovery Point Objective)** → Max acceptable data loss; drives backup frequency + replication.

- **Disaster Recovery** → Cross-region failover capability; regular DR drills required.
- **High Availability** → Automatic failover with minimal downtime; Patroni, AWS Multi-AZ, always-on AG.
- **Geo-Redundant Backup** → Backups stored in different region; protects against regional outage.
- **Backup Verification** → Automated restore test; validate checksum; "backup that's never restored is untested".
- **Backup Encryption** → Encrypt backups at rest; manage keys separately from data.
- **Keywords**: RPO drives backup frequency; RTO drives HA tier; always test restoration; PITR requires continuous WAL archiving.

---

29. Observability & Database Monitoring

Key Metrics

- **Cache Hit Ratio** → `shared_buffers` hits / total reads; target >99% for OLTP; low = add RAM or tune.
- **Replication Lag** → Bytes or seconds behind primary; alert if growing; causes stale reads on replica.
- **Connection Count** → Active + idle + idle-in-transaction; alert when approaching `max_connections`.
- **Lock Wait Time** → Time transactions spend waiting for locks; high value = contention problem.
- **Deadlock Rate** → Deadlocks per second; investigate application lock ordering.
- **Bloat** → Dead tuple accumulation in tables/indexes; causes I/O waste; indicates autovacuum issues.
- **Autovacuum Activity** → Track tables with high dead tuple counts; tune autovacuum thresholds.
- **Slow Queries** → Queries exceeding threshold; first thing to check for performance issues.
- **Wait Events** → What queries are waiting on (I/O, CPU, Lock, LWLock); PostgreSQL `pg_stat_activity`.
- **Disk Usage Trends** → Monitor growth rate; alert before full; include WAL and temp space.

Diagnostic Tools

- **EXPLAIN / EXPLAIN ANALYZE** → Query plan inspection; most important DB performance tool.

- **pg_stat_statements** → Aggregate stats per query fingerprint; find top CPU/IO-consuming queries.
- **pg_stat_user_indexes** → Track index usage; find unused indexes (drop them — write overhead for free).
- **pg_stat_activity** → Live view of running queries; identify long-running transactions.
- **Query Performance Insight** → AWS RDS/Aurora managed tool; top queries + wait events dashboard.
- **pgBadger** → PostgreSQL log analyzer; generates HTML reports from slow query logs.
- **Slow Query Log** → MySQL/PostgreSQL log queries over threshold; essential for production tuning.
- **Percona Monitoring and Management (PMM)** → Open-source MySQL/PostgreSQL monitoring stack.

Alerting

- **Alert on**: Replication lag > threshold, connections > 80% max, long-running transactions > N minutes, lock waits spiking, dead tuple ratio > threshold, disk > 80% full, cache hit ratio < 95%.
- **Plan Regression** → Query plan changes after statistics update or upgrade; compare EXPLAIN output over time.
- **Keywords**: Monitoring without alerting is useless; EXPLAIN ANALYZE is the first tool for any slow query; pg_stat_statements reveals hidden query patterns.

---

30. Security

Authentication & Authorization

- **Authentication** → Verify identity; password, certificate, IAM role, Kerberos, LDAP.
- **Authorization** → Control what authenticated user can do; GRANT/REVOKE; roles.
- **RBAC (Role-Based Access Control)** → Permissions assigned to roles; users assigned to roles.
- **ABAC (Attribute-Based Access Control)** → Permissions based on attributes (department, clearance level).
- **Principle of Least Privilege** → Grant minimum permissions needed; service accounts should have minimal rights.
- **Row-Level Security (RLS)** → Policies filter rows per user/role; `CREATE POLICY` in PostgreSQL; tenant isolation.

- **Column-Level Security** → GRANT SELECT on specific columns only; hide sensitive columns.
- **Dynamic Data Masking** → Show masked values (e.g. `XXXX-1234`) to low-privilege users; actual data unchanged.

Encryption

- **Encryption at Rest** → Encrypt data files, backups, WAL; AES-256; transparent DB encryption (TDE).
- **Encryption in Transit** → TLS/SSL for client-DB connections; enforce `sslmode=require`.
- **Key Management** → Rotate encryption keys; use KMS (AWS KMS, HashiCorp Vault); never hard-code keys.

Hardening

- **SQL Injection Prevention** → Always use parameterized queries / prepared statements; never concatenate user input.
- **Secrets Management** → Store credentials in Vault, AWS Secrets Manager; rotate automatically.
- **Auditing** → Log who accessed/modified what; `pgaudit` extension; required for compliance.
- **Privileged Access Management (PAM)** → Control and audit DBA access; just-in-time elevation.
- **Network Isolation** → DB not publicly accessible; VPC/private subnet; firewall rules; security groups.
- **Keywords**: Parameterized queries are non-negotiable; RLS is the right way to enforce tenant isolation at DB level.

---

31. Compliance & Data Regulations

Major Regulations

- **GDPR** → EU; protects personal data of EU residents; consent required; right to erasure; data portability; fines up to 4% of global revenue.
- **CCPA** → California; similar to GDPR for California residents; opt-out of data sale.
- **HIPAA** → US healthcare; Protected Health Information (PHI); encryption required; audit logs required.
- **PCI-DSS** → Payment card data; encrypt cardholder data; restrict access; quarterly scans.
- **SOC 2** → Security/availability/processing integrity/confidentiality/privacy controls; audit report.

Data Handling

- **PII (Personally Identifiable Information)** → Name, email, SSN, IP address; special handling required.
- **Data Classification** → Public / Internal / Confidential / Restricted; drives controls applied.
- **Anonymization** → Remove all identifying info; irreversible; GDPR-compliant.
- **Pseudonymization** → Replace identifiers with tokens; reversible; still personal data under GDPR.
- **Right to Erasure (Right to be Forgotten)** → Delete all personal data on request; complex with backups + event stores.
- **Data Retention Policy** → Define and enforce max storage duration; auto-purge after expiry.
- **Data Residency / Sovereignty** → Data must stay within a country/region; affects DB region choice.
- **Consent Management** → Track user consent; only process data with valid consent.
- **Audit Trail** → Immutable log of data access and changes; required by HIPAA, PCI-DSS.
- **Cross-Border Transfer** → GDPR restricts EU data transfers to non-adequate countries; Standard Contractual Clauses needed.
- **Keywords**: Anonymization is the only way to fully escape GDPR scope; pseudonymization is not enough; right to erasure is hardest in event-sourced systems.

---

32. Cloud Databases

Managed Services

- **Managed Database** → Cloud provider handles patching, backups, HA, scaling; trade control for convenience.
- **Amazon RDS** → Managed MySQL, PostgreSQL, SQL Server, Oracle; Multi-AZ for HA; Read Replicas for scale.
- **Amazon Aurora** → MySQL/PostgreSQL compatible; disaggregated storage; 6-way replication across 3 AZs; faster failover.
- **Google Cloud Spanner** → Globally distributed; strong consistency; horizontal scaling; TrueTime for external consistency.
- **Google AlloyDB** → PostgreSQL-compatible; columnar engine for analytics; ML-accelerated.
- **Azure Cosmos DB** → Multi-model; globally distributed; 5 consistency levels; turnkey multi-region writes.
- **Cloud SQL** → Managed MySQL/PostgreSQL/SQL Server on GCP.

- **Neon** → Serverless PostgreSQL; scale-to-zero; branching (Git-like DB branching for dev/test).
- **PlanetScale** → Serverless MySQL; Vitess-based; schema branching; horizontal sharding.

Cloud Patterns

- **Multi-AZ Deployment** → Synchronous standby in another AZ; automatic failover; ~1 min RTO.
- **Auto Scaling** → Aurora Serverless scales compute automatically; DynamoDB on-demand mode.
- **Read Endpoint / Write Endpoint** → Route reads to replica, writes to primary via proxy DNS.
- **Database Proxy** → RDS Proxy; pools connections; reduces failover time; important for Lambda workloads.
- **IOPS Provisioning** → Allocate dedicated I/O throughput; burst vs provisioned; gp3 vs io1/io2 on AWS.
- **Maintenance Windows** → Scheduled patching window; configure for low-traffic period.
- **Cost Optimization** → Right-size instances; use reserved instances (1-3yr); S3-backed cold storage; delete unused snapshots.
- **Keywords**: Aurora's storage is separate from compute — scale independently; RDS Proxy is essential for serverless DB connections; Multi-AZ ≠ Multi-Region.

---

33. Database Tools & Ecosystem

Migration & Transformation

- **Flyway** → Version-controlled SQL migrations; simple; SQL-first.
- **Liquibase** → XML/YAML/SQL changesets; diff-based; more features.
- **dbt** → SQL transformation layer on DWH; modular models; lineage; tests; documentation.
- **Apache Airflow** → Workflow orchestration; DAG-based ETL pipelines.
- **Great Expectations** → Data quality framework; validate data at pipeline checkpoints.

Connectivity & Pooling

- **PgBouncer** → Lightweight PostgreSQL connection pooler; transaction mode recommended.
- **ProxySQL** → MySQL proxy + pooler + router; query caching; read/write splitting.
- **Vitess** → MySQL horizontal sharding middleware; used by YouTube, Slack.
- **Citus** → PostgreSQL extension for horizontal sharding; distributed SQL.

Administration & Monitoring

- **DBeaver** → Universal DB GUI; open-source; supports 80+ DBs.
- **DataGrip** → JetBrains DB IDE; smart SQL completion; schema migration.
- **pgAdmin** → PostgreSQL-specific web UI; built-in EXPLAIN visualizer.
- **Percona Toolkit** → MySQL/PostgreSQL command-line tools; pt-osc, pt-query-digest.
- **pgBadger** → PostgreSQL log analyzer; slow query report.

Data Discovery & Governance

- **Apache Atlas** → Metadata management + governance; lineage tracking.
- **DataHub** → LinkedIn open-source data catalog; search + lineage + ownership.
- **Amundsen** → Lyft data discovery; metadata catalog; popularity ranking.
- **Data Lineage** → Track data from source to destination; critical for compliance + debugging.
- **Data Catalog** → Centralized inventory of data assets; searchable; owned.

---

34. Anti-Patterns

Query Anti-Patterns

- **N+1 Query Problem** → Loop in app code fetching one related row per iteration; use JOIN or batch fetch instead.
- **SELECT *** → Fetches unused columns; breaks covering indexes; fragile with schema changes.
- **Full Table Scan Abuse** → Query without appropriate index; no WHERE or low-selectivity WHERE.
- **Unbounded Queries** → No LIMIT; one bad query returns millions of rows and crashes app.
- **Correlated Subquery in SELECT** → Re-executes per row; use JOIN or window function instead.
- **Implicit Type Cast** → `WHERE created_at = '2024-01-01'` when column is TIMESTAMP; breaks index use.
- **ORM-Generated Inefficient Queries** → Lazy loading, `findAll()` without pagination; inspect generated SQL.

Schema Anti-Patterns

- **Missing Index** → FK columns without index; foreign key lookups cause full scan.
- **Over-Normalization** → Too many joins for simple queries; denormalize hot paths.
- **Under-Normalization** → Redundant data; update anomalies; inconsistency.

- **EAV (Entity-Attribute-Value)** → Flexible but un-queryable at scale; use JSONB instead.
- **Storing BLOBs in DB** → Images/files in DB; use object storage (S3) + store reference URL.
- **Magic Numbers / Untyped Columns** → `status = 1`; use enum or lookup table.
- **Growing Arrays** → Unbounded array columns or subdocuments; causes document/row bloat.
- **Soft Delete Without Index** → `WHERE deleted_at IS NULL` full scan; add partial index.

Architecture Anti-Patterns

- **DB as Message Queue** → Polling a table for work; doesn't scale; use Kafka/SQS instead.
- **Shared DB Across Microservices** → Tight coupling; shared schema = deployment dependency.
- **Hot Partition** → Sequential shard key causes all writes to one node; add entropy to key.
- **Single Point of Failure** → No replica, no HA; one crash = total downtime.
- **Long-Running Transactions** → Hold locks; bloat MVCC; block autovacuum; kill proactively.
- **Ignoring Query Plan Regressions** → Plan changes after upgrade/stats update; monitor with query store.
- **Keywords**: N+1 is the most common ORM problem; never allow unbounded queries in production; shared DB is the microservices anti-pattern that kills independence.

---

35. Advanced / Staff+ Level Topics

Global Scale

- **Multi-Tenant Database** → Shared schema (row-level isolation via tenant_id), shared DB (schema per tenant), or dedicated DB per tenant; each has cost/isolation trade-offs.
- **Global Replication** → Multi-region active-active or active-passive; latency vs consistency trade-off.
- **Data Residency** → Store EU data in EU; routing layer enforces data geography.
- **Active-Active** → Both regions serve reads + writes; conflict resolution required; hardest consistency.
- **Active-Passive** → One region active, one hot standby; simpler; 30–60s failover typical.
- **Conflict Resolution** → Last-write-wins (LWW), vector clocks, application-level merge; must be defined for multi-master.
- **Spanner TrueTime** → Google's globally synchronized clock using GPS+atomic clocks; enables external consistency with bounded uncertainty.

Emerging Architectures

- **Data Mesh** → Domain-owned data products; federated governance; decentralized; each domain owns its pipeline.
- **Lakehouse** → Data lake + warehouse features; open formats (Parquet/ORC) + ACID transactions (Delta Lake, Iceberg, Hudi).
- **Disaggregated Storage** → Compute and storage separated; Aurora, Socrates (Azure SQL), AlloyDB; scale each independently.
- **HTAP Systems** → Single DB for OLTP + OLAP; TiDB, SingleStore; eliminates ETL lag.
- **Tiered Storage** → Hot (SSD/NVMe), warm (HDD), cold (S3/GCS/Blob); auto-tiering by age/access frequency.
- **Serverless Databases** → Scale to zero; instant start; pay per request; Neon, Aurora Serverless v2, PlanetScale.

Advanced Internals

- **CockroachDB** → Distributed SQL; Raft per range; serializable isolation; auto-rebalancing; Postgres-compatible.
- **YugabyteDB** → PostgreSQL protocol + Cassandra API; Raft-based; multi-master OLTP.
- **FoundationDB** → Ordered KV store; ACID; used as metadata layer (Snowflake, FoundationDB Record Layer).
- **Federated Query** → Query across multiple heterogeneous DBs as if one (AWS Athena Federated, Trino, Presto).
- **Chaos Engineering for Databases** → Inject failures (kill primary, corrupt network); validate HA + recovery procedures.
- **Database SLOs** → Define and measure: p99 query latency, replication lag, availability %; tie to error budgets.
- **Cost Modeling** → IOPS cost × volume + storage tier cost + data transfer; optimize query patterns to reduce cost.

---

36. Interview Master Keywords

Trade-off Language

- **Read vs Write Optimization** → Index-heavy = fast reads, slow writes; denormalized = fast reads, complex writes.
- **Consistency vs Availability** → CAP: choose CP (HBase, Spanner) or AP (Cassandra, DynamoDB).

- **Latency vs Throughput** → Low latency needs dedicated IOPS; high throughput needs batching + parallelism.
- **Normalization vs Denormalization** → Normalize to remove anomalies; denormalize for query speed.
- **Push vs Pull Replication** → Push: primary sends; Pull: replica fetches; WAL streaming is push.
- **Synchronous vs Asynchronous Replication** → Sync = zero data loss, higher latency; Async = lower latency, possible data loss.
- **Optimistic vs Pessimistic Locking** → Optimistic = low contention, retry on conflict; Pessimistic = high contention, hold lock.
- **Fan-out on Write vs Read** → Write: precompute feed (fast read, storage cost); Read: compute at query time (slow read, storage efficient).

Decision Frameworks

- **Index Trade-offs** → Every index speeds reads but slows writes; remove unused indexes.
- **Partition Strategy** → Range for time-series; hash for even distribution; list for known categories.
- **Scaling Strategy** → Read scale first (replicas); then write scale (sharding); vertical before horizontal.
- **When to Denormalize** → After profiling shows join cost is dominant; never as a first step.
- **When to use NoSQL** → Schema flexibility, extreme scale, specific access patterns (graph, time-series, document).
- **Idempotency** → Design DB operations to be safely retried; use unique constraint or version column.
- **Backpressure** → Signal upstream when DB is overwhelmed; connection pool exhaustion = natural backpressure.
- **Hot vs Cold Data Tiering** → Move aged data to cheaper storage; query routing separates hot/cold access.

One-Liners for Interviews

- "ACID is a spectrum controlled by isolation level — higher isolation = lower concurrency"
- "CAP theorem says pick 2, but partition tolerance is mandatory — really choose CP or AP"
- "Indexes are free for reads but never free for writes — always measure the write overhead"
- "Replication lag is the gap between your consistency promise and reality"
- "Autovacuum is your friend — a database without working autovacuum will eventually fall over"
- "The N+1 problem is ORM's original sin — always check generated SQL"

- "Schema migration is a product risk, not just a dev task — expand-contract saves your deployment"
- "Eventual consistency is a spectrum — define how eventual before accepting it"
- "Connection pooling is mandatory at scale — raw connections are expensive and limited"
- "Right tool for the right job: RDBMS for integrity, Kafka for events, Redis for speed, graph DB for relationships"